NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Monday Nov. 16th by the end of class.

Portfolio 10: Searching and Sorting

Note that these activities are focused on this week's theme, but they are designed to utlize skills from all your previous portfolios as well. You should use your judgement to decide which tasks are best solved with which skills you have accumulated so far.

For completing and self-correcting 70% of the below activities, you will earn 1 grade percentage point. Additionally, for participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point*.

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Submission: To get credit for completing a prgramming activity, submit it to gradescope and answer the written questions about your algorithm, whether your program worked and what troubleshooting you can think of. You *only have to print programs you want my feedback on*. Printed programs can still be marked up by hand for syntactic or semantic corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt	2	W 11/11	W 11/11	
A2	Prog. with Pytthon (linear search)	3	W 11/11	W 11/11	
A3	Prog. with Pytthon (rec linear search)	4	W 11/11	Th 11/12	
A4	Prog. with Pytthon (rec binary search)		oyo	Th 11/12	
A5	Written (sorting game, bubble)		Th 11/12	Th 11/12	
A6	Written (sorting game, quick)		Th 11/12	Th 11/12	
A7	Review for Final		oyo	M 11/16	
A8	Review for Final		oyo	M 11/16	
A9	Review for Final		oyo	M 11/16	
A10	Review for Final		oyo	M 11/16	
B1	Share-out report	14	oyo		written
B2	Share-out report (optional)	15	oyo		written

A1) Prompt: Can computers think? Has your opinion changed? If so, what changed it? If not, what would?**

A2) Searching

1. Describe how Linear Search checks whether an item is in a list.

- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

2. Write and test a function that implements Linear Search.

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

.....

For feedback, insert you printed program(s) here.

A3) Recursive Searching

1. Describe a *recursive algorithm* for Linear Search that checks whether an item is in a list.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- What makes your algorithm recursive?

  .....

  .....

  .....

  .....

  .....

2. Write and test a function that implements Linear Search recursively.

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A4) Binary Search

1. Describe a *recursive algorithm* for Binary Search that checks whether an item (e.g. number) is in a sorted list.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- What makes your algorithm recursive?

  .....

  .....

  .....

  .....

  .....

2. Write and test a function that implements Binary Search recursively.

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A5) Sorting Game: Bubble Sort

1. Go to the sorting game website and select `Bubble Sort` from the demo menu
2. Click 'Random' to randomize the demo input
3. Start a stopwatch when you hit `Go!`
4. Stop the timer once the demo finishes sorting the list.
5. How long did the algorithm take?

   .....

6. How many comparisons were made?

   .....

7. How many Swaps were made?

   .....

8. In your own words, describe how this algorithm works.

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

A6) Sorting Game: Quick Sort

1. Go to the sorting game website and select `Quick Sort` from the demo menu
2. Click 'Random' to randomize the demo input
3. Start a stopwatch when you hit `Go!`
4. Stop the timer once the demo finishes sorting the list.
5. How long did the algorithm take?

   .....

6. How many comparisons were made?

   .....

7. How many Swaps were made?

   .....

8. In your own words, describe how this algorithm works.

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

A7) Written Questions

1. Evaluating expressions involving dictionaries

For each of the items given below, say whether they are an expression, a statement, or invalid. Interpret them in the context of variables `d1` and `d2` defined below.

- For expressions, also give their value
- For statements, also say what the effect of the statement is going to be
- For the invalid items, explain why they're invalid

```
d1 = {"tom":344, "anna":643, "peter":9482}
d2 = {"hans":633, "cay":123}
```

(a) `d1["anna"]` .....

.....

(b) `d1["hans"]` .....

.....

(c) `d1["cay"] = 737` .....

.....

(d) `d1 + d2` .....

.....

2. Basic dictionary manipulation

2.1 Give a statement that creates an empty dictionary and associates it with the variable name `new_dict`

.....

.....

.....

2.2 Give an expression that adds a new entry to the (already existing) dictionary some_dict. The key of that entry should be 5 and the value should be: `['pokemon', 'call of duty', 'guilty gear']`

.....

.....

.....

.....

A8: More Written Questions

1. Examining a function**

Consider the following function definition.

```
def mystery(alist, blist):
    if len(alist) != len(blist):
        return None
A -->d = {}
    for idx in range(0, len(alist)):
B -->   d[alist[idx]] = blist[idx]
    return d
```

1.1 What happens in the line marked with `A`?

.....

.....

.....

.....

1.2 What happens in the line marked with `B`?

.....

.....

.....

.....

1.3 In one sentence or short paragraph: what does this function do?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

2. Dictionaries and variable values

Consider the following program:

```
1.  def lists_to_dict(alist, blist):
2.      if len(alist) != len(blist):
3.          return None
4.      d = {}
5.      for idx in range(0, len(alist)):
6.          d[alist[idx]] = blist[idx]
7.      return d
8.
9.  new_dict = lists_to_dict(['a', 'b', 'c'], [100, 200, 300])
```

2.1 Draw a memory diagram that shows what we have in memory once this snippet of code has been completely executed (i.e. after line 9 has been run).

2.2 Draw a memory diagram that shows what we have in memory when line 4 has just been executed.

2.3 Draw a memory diagram that shows what we have in memory when line 6 has been executed for the first time.

A9: Review for Final

1. Fill in the blanks to make the program function.

```
print("Computing the sum of positive numbers.")

x = float(input("Enter the first number (-1 to stop)"))
sum = 0

while __1___:
    x = float(input("Enter the next number (-1 to stop)"))
    __2___

print(sum)
```

.....

.....

.....

2. What will the value of mystring be after the following code runs?

```
mystring = "3.14159265359 starts pi, while e starts with 2.7182818284"

mystring = mystring.strip("123456789\. ")
print(mystring)
```

.....

.....

.....

3. What will the value of mylist be after the following code runs?

```
def list_change(in_list):
    print(in_list)
    in_list = in_list + [4, 5, 6]

mylist = ["e", "f", "g"]
list_change(mylist)
print(mylist)
```

.....

.....

.....

4. What will the value of mylist be after the following code runs?

```
def list_change(in_list):
    print(in_list)
    in_list.extend(["a", "b", "c"])

mylist = [1, 2, 3]
list_change(mylist)
print(mylist)
```

.....

.....

.....

5. What will the value of mylist be after the following code runs?

```
def list_change(in_list):
    in_list.append("e")
    in_list.append("f")
    in_list.append("g")
    print(in_list)

mylist = [4, 5, 6]
list_change(mylist)
print(mylist)
```

.....

.....

.....

6. What will the value of mylist be after the following code runs?

```
def list_change(in_list, first_char):
    start = ord(first_char)
    for i in range(len(in_list)):
        in_list[i] = chr(start + i)*(i + 1)

mylist = [1, 2, 3]
list_change(mylist, "b")
print(mylist)
```

.....

.....

.....

7. What will the value of mylist be after the following code runs?

```
def list_change(in_list):
    in_list = in_list[1:4]

mylist = ["a", "b", "c", "d", "e", "f", "g"]
list_change(mylist)
print(mylist)
```

.....

.....

8. Write a loop or set of nested loops to print the following nested list in a grid.

```
mylist = [[1, 3, 5],
[2, 4, 6, 8, 10],
[3, 5, 7, 11],
[2, 4, 8, 16, 32, 64]]
```

.....

.....

.....

.....

.....

.....

.....

.....

9. What will the value of mylist be after the following code runs?

```
mylist = []

for i in range(5):
    mylist.append([])
    for j in range(3):
        mylist[i].append((i+1)**j)

print(mylist)
```

.....

.....

.....

.....

10. What will the following code print?

```
mylist = [["a", "b", "c"],
          ["d", "e", "f"],
          ["abc", "def", "ghi"],
          ["ball", "a", "hall"],
          "abcdefg"]

for line in mylist:
    if 'a' in line:
        print(line)
```

.....

.....

.....

.....

A10:

1. What are the three modes which can be used when opening a file?

.....

.....

.....

.....

2. What type of data does contents hold after this code runs?

```
infile = open("test_file.txt", "r")
contents = infile.readlines()
infile.close()
```

.....

.....

3. What type of data does contents hold after this code runs?

```
infile = open("test_file.txt", "r")
contents = infile.readline()
infile.close()
```

.....

.....

4. What type of data does contents hold after this code runs?

```
infile = open("test_file.txt", "r")
contents = infile.read()
infile.close()
```

.....

.....

5. What of the following is best implemented as a dictionary instead of a list?

a. The amount of time elapsed between presses of a button
b. The names on a class roster
c. The names of trees and descriptions of their leaves
d. 42 randomly generated floating point numbers

.....

.....

.....

.....

6. Which of the following is a difference between lists and dictionaries?

a. List elements cannot be mutable, but dictionary values can be.
b. Assigning to an index that does not exist in a list is an error, but assigning a value to a key that doesn't exist in a dictionary is not
c. A list can contain a dictionary as one of its elements, but a dictionary cannot contain a list as one of its value
d. There is a dict constructor that creates a dictionary from a suitable object, but there is no list constructor that does the same.

.....

.....

.....

.....

7. Will the following recursive functions always halt? Why or why not?

```
def func(num, adjust=0):
    if num % 13 ==0:
        return adjust
    else:
        return func(num+1, adjust+1)
```

.....

.....

.....

.....

```
def func(num):
    if num <= 1:
        return num
    else:
        return num + func(num+1)
```

.....

.....

.....

.....

8. Below are two examples of a recursive function to calculate Fibonacci numbers. What do you think the pros and cons of each method are?

```
def fib(num):
    if num <= 1:
        return num
    else:
        return fib(num-1) + fib(num-2)
```

.....

.....

.....

.....

```
def fib(num, val=1, prev=0):
    if num == 0:
        return num
    return fib(num-1, val+prev, val)
```

.....

.....

.....

.....

10. Remember and describe at least one of the search methods we discussed

- Linear Search
- Binary Search

.....

.....

.....

.....

.....

.....

.....

.....

11. Remember and describe at least one of the sort methods we discussed

- Bubble Sort
- Merge Sort

- Selection Sort
- Insertion Sort
- Quick Sort
- Bucket / Bin Sort

.....

.....

.....

.....

.....

.....

.....

.....

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out (optional)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....